

PID Elevator Simulator

Conversation Transcript

Building an interactive PID controller demo in a single HTML file — 30 exchanges between user and Claude

TURN 1 — USER

Build a simulator in a webpage to demonstrate how a PID controller works. The example should be an elevator and the controller tries to get the platform to the correct floor as quickly as possible. The PID terms should be broken out, along with motor power, mass in the elevator, etc. Let me know if I'm missing anything or if you have any questions.

CLAUDE

This sounds like a great project. Before I dive in, let me share what I'm planning and ask one quick thing.

****My plan:****

- Animated elevator shaft (5 floors) with the platform moving in real time
- Sliders for Kp, Ki, Kd, mass, max motor power, and friction
- Live readouts: current position, error, velocity, motor force, and each P/I/D term's contribution
- A scrolling chart showing setpoint vs. position, plus the P/I/D contributions over time
- Floor selector buttons to issue new setpoints
- Realistic physics including gravity (so the I term is genuinely needed to hold position — a classic teaching point)

****Things worth considering that you may not have mentioned:****

- ****Gravity****: makes the I term meaningful; without it, P alone can hold position
- ****Motor saturation****: a max force cap so students can see integral windup
- ****Friction/damping****: helps show how D interacts with damping
- ****Derivative on measurement**** (vs. on error) to avoid derivative kick on setpoint changes — I'll use this by default
- ****Presets****: P-only, PI, PD, well-tuned, oscillating, sluggish — good for demos

Let me check the design skill, then build it.

I've got a clear picture. Going with a dark instrument-panel aesthetic — phosphor green/amber on near-black, mono typography, oscilloscope-style charts. Let me build it.

Let me do a quick code review pass, fix a small precision issue in the settling-time calculation, and verify the file is solid before delivering it.

Let me do a quick JS syntax check to be safe:

Syntax checks out. Let me also verify the HTML is well-formed:

Clean on both counts. Copying to outputs:

Open the file in any browser — it's a single self-contained page.

****What's in there:****

- ****Elevator shaft**** on the left with 5 floors (0–4), dashed cyan setpoint line, motor + gravity force arrows alongside the car, and a SAT indicator when the motor saturates.
- ****Two oscilloscope-style charts**** in the center: position vs. time (with setpoint overlay) and the individual P/I/D contributions plus the summed motor force, with saturation limits drawn as dashed red lines.
- ****Live readouts****: position, floor, error, velocity, motor force, settling time (auto-measured: time to reach ± 0.05 m and stay there).
- ****Sliders**** for Kp, Ki, Kd, mass (200–2500 kg), motor limit (5–60 kN), viscous friction, and gravity.
- ****8 presets**** that map directly to teaching moments — P-only shows steady-state offset from gravity, PI shows oscillation without damping, Oscillating shows instability from removing Kd, Heavy Load shows how tuning breaks when mass changes, etc.

****A few implementation notes worth knowing:****

- Physics runs at 200 Hz with a fixed-timestep accumulator (stable up to high gains). Chart history samples at 60 Hz.
- ****Derivative on measurement**** ($D = -K_d \cdot \text{velocity}$), not on error — avoids derivative kick on setpoint changes.
- ****Anti-windup via clamping**** — the integral doesn't accumulate when the motor is saturated in the same direction as the error, so I-term doesn't blow up during long lifts.
- Gravity is genuinely modeled, which is what makes the I term *necessary* (otherwise a pure P controller would settle perfectly with no offset, and you'd lose the canonical PID lesson).

****Suggested demo flow if you want one:****

1. Load `P-ONLY`, send to floor 3 → watch it settle ~1 m short.
2. Load `PI-OSCILL` → it reaches the target but rings.
3. Load `WELL-TUNED` → crisp settle.
4. Load `OSCILLATE` → see why Kd matters.
5. Load `HEAVY-LOAD` then switch mass back to 800 kg without retuning → tuning is mass-dependent.

Let me know if you'd like anything tweaked — different floor count, ESP-style preset profiles, a stairstep input mode, exporting the chart data, or a "ghost trace" of a previous run for A/B comparison.

TURN 2 — USER

Move the buttons and settings to the left hand side of the page

CLAUDE

I'll use CSS `order` to flip the columns — single property change, keeps the HTML clean:

Done. New layout reads left-to-right: ****controls → charts → elevator + readouts****. The floor buttons, PID gain sliders, physics sliders, and presets are now all on the left edge.

TURN 3 — USER

No, elevator on the right, setting sliders on the left. Like split the page in half

CLAUDE

Got it — let me restructure to a true 2-column layout. All controls on the left, the elevator and its data on the right.

Now the HTML. I'll replace the entire `` block:

Now add the CSS for the right-side sub-layout (elevator + side readouts at top, charts below):

Now it's a clean 50/50 split:

- ****Left half****: target floor buttons → PID gain sliders → physics sliders → presets → notes
 - ****Right half****: elevator shaft (with state readouts + PID term bars beside it at the top) → position chart → control chart
-

TURN 4 — USER

The color scheme is currently night mode with dark background and light graphics, swap to day mode with light background and dark graphics

CLAUDE

I'll swap to a light-mode palette. Committing to a "engineering drafting paper" aesthetic — cream paper background, dark navy ink, and saturated-but-darkened trace colors that have proper contrast on light.

Let me do the swap in two steps: CSS variables + canvas-rendered colors.

Update the body texture and header gradient to suit light mode:

Now update `::selection` and all hardcoded canvas colors using a single safe non-chaining pass:

The aesthetic is now "engineering drafting paper":

- **Cream paper background** with a faint graph-paper grid texture
- **Light parchment panels** with tan borders
- **Dark navy ink** for text and the motor force trace
- **Darkened accent inks** so every series stays readable on light: forest green (position), deep teal (setpoint), burnt orange (P), deep rose (I), deep indigo (D), dark red (saturation/warnings)

Everything was repointed in one pass — CSS variables, header gradient, button glow, slider thumbs, plus all the hex codes baked into the canvas rendering for the elevator shaft and charts.

TURN 5 — USER

I want to add rag-doll physics people in the elevator. Is that hard?

CLAUDE

Not bad for stick-figure people — maybe 150 lines using verlet integration (each person = 7 points connected by distance constraints). The fun pedagogical bonus: aggressive tuning would visibly jostle them, making 'bad PID' tangible.

A full muscle-and-bone ragdoll with bendy joints and rotational dynamics gets harder fast — you'd typically pull in Matter.js or similar.

For this demo I'd recommend the stick-figure verlet route — looks great, no library, and the physics is honest (their effective gravity = real gravity + elevator acceleration, so they squash on hard liftoff and float on hard braking).

[Asked: how many passengers? style preference?]

TURN 6 — USER

2 (recommended). Stick figures (clean, fast).

CLAUDE

[Built the verlet ragdoll system: 7 points per passenger (head, shoulders, hips, 2 hands, 2 feet), distance-constraint bones, soft diagonal stabilizers to keep them upright, gravity scaled by RAGDOLL_GRAVITY_SCALE for visible response, foot friction so they don't slide. Spawned 2 passengers in the elevator car, rendered as line drawings with filled/hollow head circles for visual distinction.]

TURN 7 — USER

They fall over too easily, make the legs spread more

CLAUDE

The geometry is the problem — with leg bones rest-length 30 connecting hips to feet at ± 4 , the equilibrium pose has feet basically directly under the hips, so the base is narrow and any sideways perturbation tips them. The fix: lengthen the leg bones so feet naturally splay outward like a tripod, and add foot friction so they 'grip' the floor.

Changed three things:

1. Splayed-out legs — leg bones now have rest length 32 while the vertical hip-to-floor drop is 28, so geometry forces feet outward into a tripod (feet rest at $\sim \pm 15$ from center).
2. Stabilizer rest length 28 between feet (was 8) so the figure doesn't fight its own widened stance.
3. Foot friction — feet within 2px of the floor get 80% lateral velocity damping per step.

TURN 8 — USER

Remove the Notes section, move the position/time charts under presets, and make the elevator shaft taller, and the elevator cabin square (taller)

CLAUDE

Four changes done:

- Notes panel removed
- Position + Control charts now live in the left column right below Presets
- Shaft canvas taller: 460 $\rightarrow$ 700 px
- Cabin is now a 130 $\times$ 130 square (was 230 $\times$ 78). With the taller shaft, each floor spacing works out to ~ 130 px, so the cabin almost exactly fills one floor's worth of vertical space — visually clean.

One thing to note: the right column (elevator + state + control signal) ends up shorter than the left column. Easy fixes are to stack STATE/CONTROL SIGNAL below the shaft, or grow the shaft taller. Let me know.

TURN 9 — USER

did you update the html file?

CLAUDE

Yes — file is updated. Both copies are identical (same timestamp, same size: 49,343 bytes), the new code (configureForCanvasHeight, dynamic RAGDOLL_SCALE, min-height: 1200px) appears 10 times, and the old display-top/display-side wrappers are gone (0 occurrences).

TURN 10 — USER

The simulator page looks exactly the same, the elevator is below the controls

CLAUDE

I think I diagnosed it. The file IS updated, but the layout was collapsing to single column because the Claude artifact viewer panel is narrower than 1100px (my old breakpoint). When it collapses, everything stacks vertically — controls first, then the elevator panel at the bottom. That matches exactly what you're seeing.

Just dropped the breakpoint from 1100px → 700px so it stays as side-by-side columns unless you're on a phone-narrow viewport. You should now see the two-column layout.

TURN 11 — USER

The physics of the stick figures doesn't seem correct. Like they are relative to the elevator not the world

CLAUDE

You're right — my current implementation simulates in the car's reference frame (with a pseudo-force tacked on for elevator acceleration). It's mathematically equivalent to world-frame physics in steady state, but visually it gives a uniform, glued-on feel: all body parts experience the same effective gravity at once, so there's no 'wave' of motion from the feet pushing the body up.

In a proper world-frame simulation, the elevator floor moves first, the feet get pushed up by contact, and the body inertia means the torso/head lag behind, propagating through the bones — that's the visible reaction you're missing.

[Rewrote the passengers to live in world coordinates: just gravity, and a moving floor/ceiling. No pseudo-forces. Now ragdoll points have world positions that don't move when the elevator moves; the cabin floor and ceiling are 'moving walls' in the world that push the ragdoll through containment.]

TURN 12 — USER

Add a red particle spray at any joint experiencing a force greater than 20g

CLAUDE

Detection approach: after each physics step, check how much constraint solving + containment displaced each joint from its naturally-extrapolated trajectory. That displacement, divided by dt^2 , gives the implied acceleration. If it exceeds 20g, that's a high-force event and I spawn a red spray at that joint.

Brief math: A point's 'naturally expected' next position after one verlet step is $pos + prev_velocity \cdot dt - g \cdot dt^2$. Constraints (bones) and containment (floor/ceiling/walls) then push it from there. The total constraint-imposed displacement Δp satisfies $\Delta p = a_constraint \cdot dt^2$, so the threshold $|\Delta p|^2 > (20g \cdot dt^2)^2$ cleanly maps to *implied acceleration exceeds 20g*.

TURN 13 — USER

Compress the UI vertically. It's currently about twice as tall as my screen. The elevator can be smaller, but I also think the sliders can be more compressed. Remove the presets window. Make the plots less

tall

CLAUDE

Done. Estimated left column height after changes:

- Targets ~50, PID ~145, Physics ~180, State ~155, Control Signal ~130, each chart ~175, gaps ~85 = roughly 1100px total (down from ~1700)

The elevator min-height is now 600px and it flexes to match the left column.

TURN 14 — USER

Make the sliders shorter and put the name and value on the sides rather than above. The elevator can be a little taller.

CLAUDE

Sliders are now single-row: label on the left in an 80px column, slider stretches in the middle, value right-aligned in 70px. That cuts each slider from ~38px to ~22px tall — PID Gains panel goes from ~145 → ~90, Physics from ~180 → ~115, so the left column should be ~120px shorter overall. The elevator min-height also bumped 600 → 720 for a slightly taller shaft.

TURN 15 — USER

Move the floor buttons to the top of the elevator instead of top of the left half of the page.

CLAUDE

Done. Floor button panel is now the first thing in the right column, sitting just above the elevator shaft. The elevator panel still uses flex: 1 so it'll shrink slightly to make room for the buttons, but since the left column drives the row height, both columns will still align.

TURN 16 — USER

change the size of the buttons and elevator to fit it all on one page

CLAUDE

Smaller buttons + paired PID/Physics and STATE/CONTROL SIGNAL panels side-by-side + shorter charts (90px canvas) + lower elevator min-height. Estimated layout now:

- Pair 1 (PID GAINS | PHYSICS): ~140px tall
- Pair 2 (STATE | CONTROL SIGNAL): ~140px tall
- Position chart: ~130px
- Control chart: ~130px

Total left column: ~580px (down from ~950).

TURN 17 — USER

[Uploaded screenshot showing broken slider layout]

Side by side slider layout doesn't work. It's fine for State and Control Signal windows, but not the variable inputs. The elevator shaft can be taller now as well.

CLAUDE

You're right - the sliders need their own row width. Undid the panel-pair for PID GAINS and PHYSICS — each gets its own full-width row again. Kept STATE+CONTROL SIGNAL paired (they fit fine side-by-side). Elevator min-height bumped to 560.

TURN 18 — USER

Make the elevator shaft auto-scale to the window

CLAUDE

Done. The page now locks to viewport height:

- body is a flex column at height: 100vh with no overflow
 - main flexes to fill the space below the header
 - Right column (col-display) overflows hidden, with the elevator panel using flex: 1 to consume whatever vertical space the column has
 - Left column (col-controls) gets overflow-y: auto as a safety net so it scrolls internally if the controls exceed the window height instead of pushing the elevator off-screen
 - Removed the min-height: 560px from the elevator panel so it shrinks freely with the window
 - Resize handler now also respawns passengers so they're at the correct scale after a resize.
-

TURN 19 — USER

Write a < 350 character description for github repo

CLAUDE

Interactive PID controller playground rendered as an elevator: tune Kp/Ki/Kd against a physics-simulated motor under gravity and friction, watch verlet-ragdoll passengers jostle, lift off, and squash in response to bad tuning, and trace position and control signals on live charts. Single-file HTML, no dependencies.

(345 characters)

TURN 20 — USER

Auto scale the two plots depending on the data floor and ceiling

CLAUDE

Both charts now auto-fit:

- Position chart scans the visible history window for min/max of position and setpoint, adds 10% padding (minimum 0.3m), and snaps to nice round numbers. Floor reference lines and the readout dot only draw if they fall inside the current view.
- Control chart finds the min/max across all four series (P, I, D, Σ Motor), adds 8% padding (minimum 100 N), and snaps similarly. Motor saturation lines only render when they're inside the current Y range.

Also fixed a bug where the chart canvases were using a hardcoded $h=240$ even though the actual canvas height is much shorter — now reads `clientHeight` so labels and gridlines line up correctly.

TURN 21 — USER

For the ragdolls, lets add some springiness to the joints, currently the figures are stiff minus the arms swinging. On impacts the head and legs should flex (rotate) based on external forces.

CLAUDE

Three changes:

1. Knees added. Each leg is now hip → knee → foot instead of a single rigid rod. The knees start biased slightly outward so they tend to flex out rather than buckle inward under compression. A soft hip-foot stabilizer ($k=0.12$) pulls the leg back toward straight at rest.
2. Stabilizers softened. Spine (head-hips) and head-foot diagonals dropped from $k=0.25-0.30$ to $k=0.05-0.08$. The body now sways and the head bobs noticeably during acceleration, instead of staying glued to the upright pose. Neck bone also relaxed ($0.5 \rightarrow 0.35$) so the head springs around lightly.
3. 20g detector rewritten to track only displacement from wall contact, not internal bone tension. Each step accumulates the per-iteration containment correction at each joint; only that displacement is checked against the threshold. This means soft springy joints in their normal 'fighting gravity' mode no longer trigger false positives — only actual collisions with the cabin floor/ceiling/walls do.

TURN 22 — USER

Soften all the springs, the movement isn't very noticeable.

CLAUDE

Roughly halved all the stiffnesses:

Neck (head-shoulders): $0.35 \rightarrow 0.20$

Torso: $0.50 \rightarrow 0.30$

Arms: 0.45 → 0.25

Legs (each segment): 0.45 → 0.27

Spine stabilizer: 0.08 → 0.04

Head-foot diagonal: 0.05 → 0.025

Leg-straightness: 0.12 → 0.06

Plus damping reduced 0.99 → 0.994 so the springs ring longer before settling. You should see clearly visible body sway, head bob, knee buckle, and torso compress/stretch on impacts.

TURN 23 — USER

Remove the second plot, and add a toggle to the first plot to switch between position and PID plots, then auto-scale the height of the plot to fill to the bottom of the page

CLAUDE

Done:

- Second chart removed, both are unified into one canvas (#chart-main)
- Toggle tabs in the chart header — 'POSITION' / 'CONTROL'. Active tab inverts to filled style. Meta text and legend swap automatically based on the selected view.
- Chart panel uses flex: 1 to fill the remaining left-column height. `resizeCanvases` measures the panel and subtracts header + legend height to compute the canvas size. So as you resize the window, the chart canvas auto-stretches to the bottom of the page.
- Switching views also triggers a resize, since the legend height can change between views (2 items vs 4 items might wrap differently on narrow viewports).

TURN 24 — USER

Create a PDF of this entire chat using the format guide

CLAUDE

[Built this PDF by parsing the transcript file for the early conversation and appending the post-compaction exchanges from the current context. Followed the format guide: Helvetica typography, blue-background user messages, regular assistant text, gray turn labels, thick rules between header/footer and thin rules between turns.]

TURN 25 — USER

Add a button below 'target floor' section to add or remove passengers to the elevator

CLAUDE

New PASSENGERS panel added below TARGET-FLOOR with [-] [count] [+] buttons. Count is decoupled from elevator mass — passengers are still purely visual ragdolls. Each add/remove respawns all passengers evenly distributed across the cabin interior at the current floor height, alternating filled/hollow head variants. Buttons disable at 0 and at the initial cap of 6.

TURN 26 — USER

Limit it to 99 passengers

CLAUDE

Cap raised from 6 to 99. Heads up: each passenger has 9 verlet points $\times$ 6 constraint iterations per 200Hz physics tick, so past about 30-40 you'll feel a frame-rate hit, and the 2.5m cabin gets very crowded with the even-distribution math packing slots tighter as N grows.

TURN 27 — USER

Do the passengers add mass to the system?

CLAUDE

No. Passengers are purely visual ragdolls right now — they react to the cabin floor moving but don't feed back into state.mass. The elevator's mass is whatever the Physics slider says, regardless of passenger count.

Want them linked? A simple approach would be $\text{effective_mass} = \text{base_mass} + \text{passengerCount} \times 75\text{kg}$, so adding people makes the elevator visibly harder to lift and easier to overshoot when descending.

TURN 28 — USER

Yes, have the passengers add mass to the base

CLAUDE

Each passenger now adds 75 kg to the effective mass used in both gravity ($F_g = -M \cdot g$) and Newton's second law ($a = F_{\text{net}} / M$). The MASS slider represents the empty cabin; a new LOAD row in the STATE panel shows the additional passenger mass live (+150 kg for 2, +7425 kg for the full 99). The motor works noticeably harder, the integral term grows, and overshoot gets more pronounced as the cabin fills.

TURN 29 — USER

What about adding the force of the bodies instead of just the mass? So when they are floating separate from the elevator the load decreases but when they crash into the floor it increases

CLAUDE

Much richer physics — switching from static-mass coupling to true reaction-force coupling.

The containment loop already accumulates per-point displacement (cdy) from the cabin floor/ceiling each step. By Newton's 3rd law that IS the reaction force passengers exert on the elevator: if a point gets clamped UP by the floor by cdy pixels, the floor pushed it with $F = m \cdot cdy/dt^2$ (pixel units), so the point pushes the floor DOWN with that same force. Sum across all points, convert to Newtons, add to the elevator's F_{net} .

Behavior this produces:

- Steady standing → load = passenger weight (matches the old approach in steady state)
- Hard upward accel → load > weight (compression on floor)
- Hard upward decel during ascent → passengers fly up, hit ceiling → load goes negative (they're pulling the cabin UP, fighting the deceleration → overshoot worsens)
- Free-fall regime → load → 0 (the elevator suddenly feels light and is harder to brake)

MASS slider is again just the empty cabin's inertia. The LOAD readout now shows the live (exponentially-smoothed) load in Newtons, lighting warn-red when load exceeds $\sim 1.8\times$ the static expected weight (impact events).

TURN 30 — USER

Generate an updated transcript pdf

CLAUDE

[This PDF — appended the passenger button, 99-cap, mass-coupling, and force-coupling exchanges to the post-compaction list and rebuilt with the same ReportLab formatting as before.]